QUANTUM 开发者说明（架构规则速记）

QUANTUM = Quadtree Unified Adaptive Numerics for mulTiphysics with eMbedded geometry

本说明只覆盖“程序架构与配置规则”，用于提醒与对齐团队实现习惯（非算法文档）。

1. 分层原则（从底到顶）

- Core：网格/几何/场/插值/IO 等“基础核”，不包含特定物理流程。
- Solvers：FV、IIM/EB、线性求解器封装等可复用数值算子与求解器。
- Modules：基础物理能力（Electricity/Hydro/PIC 等）的“可独立运行模块”。每个模块只拥有一个模块字典文件。
- Apps：应用工作流（Electrospray、验证回归等）。负责模块串联、宏时间步、重启、耦合开关等。
- Cases：发布级算例输入目录；cases 只提供字典与初始文件，不包含脚本逻辑。

2. 目录规范（用户视角优先）

推荐目录（发布级）

```
QUANTUM/
 apps/
  quantum_main.cpp
  include/Runtime/...
  src/Runtime/...
    Apps/
     ElectrosprayApp.*
     PicVerificationApp.*
     SingleModuleApp.*
 include/
  Core/..., Solvers/..., Modules/...
 src/
  Core/..., Solvers/..., Modules/...
 cases/
  <caseName>/
   system/（仅字典文件）
   output/
   cache/
```

规则：运行组织层（RunController/AppRegistry/ModuleRegistry 与 App 实现）统一放在 apps/ 下；算法与数据结构留在 include/src。

3. system/ 字典文件规则（强约束）

允许文件（最小集合）

```
 mesh.dict
 geometry.dict
 run.dict       # 统一控制文件：选择 app + App 工作流配置（已与 app.dict 合并）
 electricity.dict # 可选：当该 case 会用到 ElectricityModule
 hydro.dict      # 可选：当该 case 会用到 HydrodynamicsModule
 pic.dict       # 可选：当该 case 会用到 ExplicitPICModule
```

不再使用 app.dict；工作流配置写在同一个 run.dict 中，按 “ AppName 块 ” 存放。run.dict 不出现 module/modules。

run.dict 合并版示例

```
Run
{
 app SingleModule
}

SingleModule
{
 modules ( Hydrodynamics Electricity ExplicitPIC )
}

Electrospray
{
 macroDt    1e-7
 finalTime  1e-4
 restart    false
 couplePIC  true
}
```

4. 运行时如何选择 Case 与 App

- Case 选择（优先级）：优先使用命令行 --case <path>；否则若当前目录存在 ./system/ 则视为 case 根；再否则回退到 ./cases/default。
- Case 选择（TTY 交互）：若未使用 --case 且当前目录不是 case 根（无 ./system/），并且 stdin 为 TTY，则在 ./cases/ 下列出可用 case 供选择（回车使用默认 case，例如 default）。非交互运行时不提示，直接回退默认 case。
- App 选择（由 run.dict 决定）：进入选定 case 后，读取 system/run.dict 的 Run{ app ... }，不再在控制台选择 app。
- App 工作流：仅由 App 解释其同名块（如 Electrospray{...}）；模块串联顺序等不由 run.dict 的通用字段决定。

5. 新增 App 的最小步骤

1. 在 apps/src/Runtime/Apps/ 新建 <MyApp>App.cpp，并在 apps/include/Runtime/Apps/ 提供对应头文件。
2. 实现接口：name() 与 run(...)（按当前 IApp 约定）。
3. 在 AppRegistry 注册别名：建议支持 MyApp 与 MyAppApp 两种名字。
4. 若需要工作流参数：在 case 的 run.dict 中加入同名块 MyApp{...}，并在 app 内解析该块。
5. App 内部按需调用基础 Modules（Electricity/Hydro/PIC），基础模块仍应可独立运行。

6. 新增 Module 的最小步骤

1. 在 include/Modules/<ModuleName>/ 与 src/Modules/<ModuleName>/ 添加实现。

2 每个模块最多一个模块字典文件：system/<module>.dict（例：electricity.dict/hydro.dict/pic.dict）。

3 模块应支持被 SingleModule app 单独运行（便于调试与复用）。

4 模块不读 run.dict 的通用字段；模块只读自己的模块 dict。

7. 发布卫生（Release Hygiene）

- cases 目录不包含脚本逻辑；运行切换通过 --case 与 TTY 下的 case 选择完成。
- 目录/文件命名不出现 Task/Phase 等对话过程的临时名称。
- 输出建议统一写到 case 的 output/ 子目录，避免污染仓库根。
- 新增功能优先以“新增 App/Module”方式扩展，而不是在 main 中硬编码流程。

附：常见问题速查表

问题	规则/建议
在哪里选择运行哪个 App？	在 case 的 system/run.dict 的 Run{ app ... }；运行时不再交互选择 app。
如何在控制台选择 Case？	在 TTY 交互模式下，若未提供 --case 且当前目录不是 case 根，程序会列出 ./cases/ 下可用 case 供选择；直接回车使用默认（如 default）。
在哪里配置模块串联顺序？	写在 run.dict 的 SingleModule{ modules (...) }（或对应 AppName 块）中，由 App 解析。
模块参数放哪里？	放在模块自己的 dict：electricity.dict/hydro.dict/pic.dict。
根目录调试默认用哪个 case？	未给 --case 且当前无 ./system 时回退到 cases/default（TTY 下会先提示可选 case）。